Story Point Estimation with BERT Word Embeddings - Final Report

PS-3: Evaluate BERT-based word embeddings for story point estimation when training on source projects and testing on a different target project under the assigned random-split protocol.

Group 12 - Challa Gouri Shankar (B230648CS), Konki Navadeep (B230386CS), E Srinith Reddy (B230929CS), Fahad Ahmed Mahdi (B230103CS), Anita (B230818CS)

GitHub: https://github.com/ettasrinith/story-point-estimation

#1. Problem

Agile teams label each issue with a story point - a rough measure of the work it needs. We study whether a model that reads the issue text can predict this number, and specifically whether such a model transfers across projects: train on one project's issues, test on a different project it never saw. This is repeated over the 39 training/testing pairs assigned in the project list, using BERT word embeddings as the text representation, a three-layer CNN as the predictor, random 80:20 splitting, and MAE / MdAE as the metrics.

#2. Dataset

TAWOS (Tawosi et al., MSR 2022; DOI 10.5522/04/21308124) - 65427 Jira issues with story points across 39 open-source projects (extracted from the official MySQL dump by 01_extract_tawos.ipynb; per-project counts in results/manifest.csv).

Cleaning: story points cast to numeric, issues without a story point dropped (458,232 -> 65,427), HTML markup stripped from title + description, duplicates removed. For all experiments story points are kept in (0, 40] - this removes zero-effort tickets and broken values (e.g. one project contains a story point of 26,710,114) - leaving 62,193 issues. Project sizes range from 209 (Apache MXNet) to 19,358 (Lsstcorp Data management) issues.

#3. Method

Representation. bert-base-uncased, frozen, inference only. Each issue's text (title + description) is tokenized (max length 128, ~13% of the longest descriptions truncated) and encoded once; the token-level last-hidden-state vectors [128 × 768] are stored per project (float16, 12.2 GB total; 03_bert_embeddings.ipynb).

Model. Three-layer 1D CNN (768->256->128->64 channels, kernel 3, ReLU, dropout 0.2 on the first layer) over the token embeddings, masked global max pooling over real tokens, then a small MLP head (64->32->1) predicting the story point. L1 loss (matches MAE), Adam lr 1e-3, batch 64, early stopping on validation MAE (patience 4, max 25 epochs). fp16 autocast on a T4 GPU.

Protocol (per pair). Each project's issues are split randomly 80:20 with seed 42 - the same partitions used by the baseline, so results are directly comparable. (1) 5-fold cross-validation on the source project's 80% pool: train on 4 folds, validate on 1; reports mean +/- std validation MAE.

(2) The number of epochs is fixed to the folds' average, and a final model is trained on the full pool. (3) Single evaluation on the target project's held-out 20%. The target project's data never touches training or model selection.

Baseline. Predict the source project's median story point for every target issue (02_baseline_median.ipynb).

Two project names in the assignment table have no exact match in the dataset ("Evergreen", "C++ driver (Appcelerator)"); with the instructor's approval they are mapped to the two remaining TAWOS projects, "The MongoDB Engineering" and "Aptana Studio".

#4. Results

Across the 39 assigned pairs:

Metric	BERT + CNN	Median baseline
Test MAE (mean +/- std over pairs)	2.326 +/- 1.350	2.496 +/- 1.450
Test MdAE (mean over pairs)	1.710	1.872
Mean 5-fold validation MAE	1.923 +/- 0.213	-
Pairs where CNN beats baseline	27/39	-

Per-pair results (validation from 5-fold CV on the source; test = held-out target):

Source	Target	Val MAE (5-fold)	Test MAE	Test MdAE	Baseline MAE
XD	MESOS	1.923 +/- 0.027	1.572	1.139	1.538
Usergrid	MXNet	0.887 +/- 0.103	1.582	0.942	2.425
Titanium Mobile Platform	Usergrid	2.363 +/- 0.308	2.346	1.688	0.729
The Titanium SDK	Alloy Framework	2.948 +/- 0.212	1.757	1.419	2.180
Software Cloud	Appcelerator Daemon	2.339 +/- 0.203	7.176	4.598	3.375
Sawtooth	Appcelerator Studio	0.899 +/- 0.084	3.231	2.866	3.646
Nexus	Command-Line Interface	0.973 +/- 0.048	1.874	1.191	2.789
MXNet	The Titanium SDK	3.281 +/- 0.908	3.149	1.558	3.470
Mule APIkit	Titanium Mobile Platform	1.782 +/- 0.304	4.287	3.065	3.168
Mule	C++ driver (Appcelerator)	2.798 +/- 0.136	3.765	2.590	5.880
Moodle	Atlassian Software Server	7.074 +/- 0.433	4.930	3.354	2.492
MongoDB Core Server	Bamboo	1.402 +/- 0.186	0.901	0.692	1.277
MESOS	Clover	1.476 +/- 0.121	3.693	1.374	4.178
Lyrasis Dura Cloud	Confluence Cloud	1.105 +/- 0.232	2.025	2.023	2.390
Lsstcorp Data management	Confluence Server	3.265 +/- 0.050	1.333	1.098	1.039
Jira Server	Crowd	1.461 +/- 0.226	1.586	1.753	2.558
Jira Cloud	FishEye	1.337 +/- 0.215	1.478	1.484	2.169
Java driver	Jira Cloud	0.085 +/- 0.050	2.990	3.131	3.243
Indy SDK	Jira Server	1.528 +/- 0.087	3.613	1.189	1.987
Indy Node	Software Cloud	1.394 +/- 0.148	1.503	1.181	2.237

FishEye DotNetNuke Platform 1.037 +/- 0.138 0.890 0.633 1.040
Fabric Blockchain Explorer 1.654 +/- 0.101 1.176 0.900 1.501
Evergreen Fabric 0.756 +/- 0.031 0.970 0.821 1.752
DotNetNuke Platform Indy Node 0.900 +/- 0.044 1.699 1.066 1.956
DataCass Indy SDK 2.403 +/- 0.411 2.383 1.916 1.620
Crowd Sawtooth 0.845 +/- 0.108 1.165 0.729 1.368
Confluence Server Lsstcorp Data management 1.306 +/- 0.113 3.177 1.540 3.886
Confluence Cloud Lyrasis Dura Cloud 1.325 +/- 0.196 1.051 0.984 1.878
Compass MongoDB Core Server 1.374 +/- 0.127 1.243 1.133 1.490
Command-Line Interface Compass 1.842 +/- 0.219 1.253 1.013 1.300
Clover C++ driver (MongoDB) 3.016 +/- 0.334 2.122 1.784 0.917
C++ driver (Appcelerator) Evergreen 3.965 +/- 0.399 3.567 3.403 6.099
C++ driver (MongoDB) Java driver 0.482 +/- 0.102 0.549 0.103 0.000
Blockchain Explorer Moodle 1.226 +/- 0.173 1.619 1.392 6.858
Bamboo Mule 1.309 +/- 0.237 2.285 1.279 3.150
Atlassian Software Server Mule APIkit 2.281 +/- 0.213 1.872 1.537 1.882
Appcelerator Studio Nexus 2.245 +/- 0.051 4.241 4.152 3.390
Appcelerator Daemon DataCass 4.899 +/- 0.960 2.579 2.141 2.401
Alloy Framework XD 1.798 +/- 0.259 2.066 1.852 2.079

Best pairs: C++ driver (MongoDB)->Java driver (0.549), FishEye->DotNetNuke Platform (0.890), MongoDB Core Server->Bamboo (0.901).
Worst pairs: Software Cloud->Appcelerator Daemon (7.176), Moodle->Atlassian Software Server (4.930), Mule APIkit->Titanium Mobile Platform (4.287).

Demo pair walkthrough (Nexus -> Command-Line Interface, 04_cnn_model.ipynb): 5-fold val MAE 0.973 +/- 0.048, test MAE 1.874 vs baseline 2.789.

#5. Discussion

- **The CNN beats the median baseline on 27 of 39 pairs and lowers the average cross-project** test MAE from 2.496 to 2.326 (improvement of 6.8%). BERT word embeddings clearly carry effort-relevant signal that transfers across projects.
- **Validation MAE (~1.92) is much lower than test MAE: the model is accurate** *within* the source project but degrades on unseen projects. The transfer gap is the central finding of this setting.
- **Worst pairs combine a source and target with very different story-point scales (e.g. a source with** median 1-2 and a target with median 5); text alone cannot recover the target's scale, so the model inherits a systematic offset. Best pairs share vocabulary domains and scales.
- **Validation results are reported separately from held-out test results, per §3.4; the final** configuration (epochs) was selected using development folds only.

#6. Threats to validity

- **Construct: story points are team-specific estimates, not objective effort; the (0, 40] filter** and per-project scales make cross-project MAE hard to interpret in absolute terms.

- **Internal: fixed seeds and partitions for all methods; model selection used development folds** only; the target project never influenced training. The two instructor-approved name mappings ("Evergreen" -> "The MongoDB Engineering", "C++ driver (Appcelerator)" -> "Aptana Studio") are the only interpretive steps in the pair list.
- **External: 39 pairs from one dataset (open-source Jira projects); results may not generalize to** commercial trackers or other estimation scales.
- **Statistical: one run per pair with a fixed seed; fold variance is reported, but no repeated** runs with different global seeds.

#7. Conclusion

BERT word embeddings with a small CNN improve cross-project story point estimation over a median baseline on 27/39 assigned pairs (test MAE 2.326 vs 2.496), but a large project-transfer gap remains. Representation quality helps; scale mismatch between projects is the dominant remaining obstacle - combining text with target-project calibration is the natural next step.

#Reproducibility

All notebooks run on Kaggle: 01 (data extraction, CPU), 02 (baseline, CPU), 03 (embeddings, T4), 04 (demo pair, T4), 05 (39 pairs, T4). Seeds, split procedure and configurations are fixed inside the notebooks; results CSVs are committed under results/.